

ANÁLISIS FORENSE DE TRANSACCIONES Y MEDICIÓN DE GAS

Leer la cadena como un investigador: qué pasó, quién pagó, cuánto y por qué.
Después, predecir el costo de cada instrucción y comprobarlo midiendo.

MODALIDAD	DURACIÓN	RED	ENTREGA
En parejas	60 min en clase + informe en casa	Sepolia (lectura) y red local	Informe de 2 páginas antes de la S6

CÓMO LEER ESTA GUÍA

Está escrita para que puedan hacer el laboratorio **sin ayuda de nadie**, incluso si nunca han abierto un explorador de bloques. Cada paso dice exactamente dónde hacer clic o qué escribir, qué deberían ver y cómo saber que funcionó. Si algo no sale como se describe, vayan a la sección «**Errores frecuentes**». Hoy no se firma ni se gasta nada: la parte A solo **lee** la cadena y la parte B corre en una red local dentro de su computador.

01 Qué van a hacer, y por qué

LEER PRIMERO

Objetivo: leer en un explorador el costo real de una transacción, descomponerlo en sus partes (tarifa base quemada y propina del validador), interpretar el campo `data` y los eventos, y comprobar con mediciones propias por qué unas instrucciones de la EVM cuestan miles de veces más que otras.

Resultado de aprendizaje: RA2 (arquitectura y costos) y, de forma complementaria, RA3 (base para programar contratos). Es una de las cuatro evidencias de laboratorio del primer corte.

PARTE	TIEMPO	QUÉ HACEN	LÁMINAS DE LA CLASE
0 · Billeteras	5 min	Punto de control: la billetera de la Sesión 2 funciona y tiene fondos de prueba.	B.1
A · Forense	30 min	Encontrar tres transacciones reales de Sepolia y llenar una hoja de trabajo por cada una.	A.9 a A.15, A.21
B · Gas	25 min	Predecir el gas de 12 operaciones, medirlo con un script y explicar las diferencias.	A.16 a A.19
Informe	en casa	Dos páginas con la plantilla de la sección 10.	C.1

REGLAS DE SEGURIDAD · NO TIENEN EXCEPCIÓN

- Solo redes de prueba. La billetera del curso **nunca** se usa con dinero real.
- La frase de recuperación no se comparte con nadie: ni con el compañero, ni con el docente, ni con un «soporte técnico».
- Hoy no hay que firmar nada. Si algo les pide una firma durante este laboratorio, **no la den** y avisen.
- Ninguna clave ni URL de nodo se escribe en un archivo del repositorio. Hoy no hacen falta.

02 Qué se entrega

PARTE	CONTENIDO	PESO
A · forense	Tres hojas de trabajo (una por transacción) con el hash como enlace que abre, las sumas comprobadas y las cuatro preguntas de la sección 05.6 respondidas.	50 %
B · gas	La tabla de 12 operaciones con predicción, medición y diferencia, y la explicación de las dos diferencias más grandes.	35 %
Reflexión	Un párrafo: después de hoy, ¿qué operación evitarían en un contrato real y qué harían en su lugar?	15 %

Un informe por pareja, **máximo dos páginas**, con la plantilla de la sección 10. Se sube a la plataforma del curso antes de la Sesión 6.

POLÍTICA DE LABORATORIOS

Sin hashes de transacción que abran en `sepolia.etherscan.io`, la parte A no se califica. Una transacción inventada —por ejemplo, por un asistente de IA— no existe en la cadena, y eso se comprueba en diez segundos.

03 Preparación desde cero

ANTES DE LA CLASE SI ES POSIBLE

La parte A solo necesita un navegador. La parte B necesita **Node.js 22 LTS** y el repositorio del curso `laboratorios-evm`, instalado una sola vez en el semestre.

1

COMPROBAR · NODE.JS Y NPM

¿Está instalado Node.js?

Abran una terminal (Windows: tecla Windows, escriban `PowerShell`, `Enter` · macOS: `Terminal` · Linux: `Ctrl + Alt + T`) y escriban:

```
node --version
npm --version
```

Salida esperada: algo como `v22.12.0` y `10.9.0`. El primer número de Node debe ser **22**.

Si dice «no se reconoce»: instalen la versión LTS desde `nodejs.org`, cierren la terminal y ábranla de nuevo.

2

UNA SOLA VEZ · INSTALAR EL REPOSITORIO

Dependencias del proyecto

Entren a la carpeta del repositorio y descarguen las dependencias:

```
cd ruta/a/laboratorios-evm
npm install
```

Salida esperada: tarda uno o dos minutos y termina con una línea como `added ... packages`. Las advertencias en amarillo (`npm warn`) no son errores.

Cómo sé que funcionó:

```
npx hardhat --version
```

debe responder `3.16.0`. Las versiones están congeladas en `versiones.md`: no se actualizan a mitad de semestre.

3

EL EXPLORADOR

Abrir el explorador correcto

Abran `https://sepolia.etherscan.io`. **Miren la dirección:** tiene que decir `sepolia`. al principio. El explorador de la red principal (`etherscan.io` a secas) se ve casi igual, y en él sus transacciones de prueba no existen. Es el error número uno del laboratorio.

La Sesión 6 despliega un contrato en Sepolia. Sin billetera con fondos de prueba, no se puede. Por eso se revisa hoy, no allá.

#	QUÉ HACER	CÓMO SÉ QUE ESTÁ BIEN
1	Abrir la billetera creada en la Sesión 2 y desbloquearla con su contraseña.	Ven su dirección, que empieza por <code>0x</code> .
2	Elegir la red Sepolia en el selector de redes. Si no aparece, activar «mostrar redes de prueba» en la configuración de la billetera.	El selector dice Sepolia , no «Ethereum Mainnet».
3	Mirar el saldo.	Hay ETH de prueba (SepoliaETH). Si es cero, avisar al docente: se reparte desde la billetera institucional.
4	Confirmar que la frase de recuperación está en papel, fuera del computador.	Saben dónde está el papel. No la escriben en ningún lado ahora.
5	Copiar su dirección y pegarla en <code>sepolia.etherscan.io</code> .	Aparece su saldo y, si recibieron fondos, la transacción de llegada. Esa es su transacción 1.

QUIEN PERDIÓ LA BILLETERA LA REHACE HOY

Creen una nueva siguiendo el procedimiento de la Sesión 2, anoten la frase en papel y pidan fondos al docente. No esperen a la Sesión 6. Y si alguien les pide la frase «para revisarla», es un intento de robo, aunque venga de alguien conocido.

Van a encontrar **tres transacciones reales de tipos distintos** en Sepolia y a llenar una hoja de trabajo por cada una. No pueden usar las de los ejemplos de esta guía ni las de las láminas: cada pareja trae las suyas.

05.1 · Cómo encontrar cada transacción

1

TRANSACCIÓN 1 · TRANSFERENCIA DE ETH

Un pago simple, sin contrato

Dónde: la página de su propia dirección (paso 5 del punto de control). En la lista *Transactions*, la fila con la que les llegaron los fondos de prueba.

Si no tienen ninguna: abran cualquier bloque reciente (en la portada del explorador, *Latest Blocks* → un número de bloque → *Transactions*) y busquen una fila cuyo *Method* diga *Transfer* y cuyo *To* no tenga el ícono de documento (el ícono indica contrato).

Cómo sé que es la correcta: al abrirla, *Input Data* dice `0x` (vacío) y *Value* es mayor que cero.

2

TRANSACCIÓN 2 · TRANSFERENCIA DE UN TOKEN ERC-20

Mover tokens es llamar a un contrato

Dónde: busquen en el explorador uno de estos dos contratos de prueba, ambos verificados:

· LINK de prueba: `0x779877A7B0D9E8603169DdbD7836e478b4624789`

· USDC de prueba (Circle): `0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238`

En la página del contrato, pestaña *Transactions*, elijan una fila cuyo *Method* sea *Transfer* y ábranla. (La pestaña *Token Transfers* también sirve, pero ahí aparecen además transferencias hechas por otros contratos; para este ejercicio queremos una llamada directa a *transfer*.)

Cómo sé que es la correcta: *To* es el contrato del token, *Value* es 0 ETH, y el *Input Data* empieza por `0xa9059cbb`.

3

TRANSACCIÓN 3 · UNA QUE FALLÓ

Revertida, pero cobrada

Dónde: en la lista de transacciones de un contrato con mucho uso (los dos tokens anteriores sirven, o cualquier contrato que vean activo en un bloque reciente), busquen una fila con el **ícono rojo de error** junto al hash. Si en la primera página no hay ninguna, pasen a la siguiente o prueben con otro contrato.

Cómo sé que es la correcta: al abrirla, *Status* dice *Fail*, casi siempre con un motivo: *Fail with error* `'...'` o *Fail With Custom Error* `'...'`.

05.2 · Qué mirar, campo por campo

Al abrir una transacción, la vista *Overview* muestra los campos principales. Hagan clic en «**Click to show more**» (o «More Details») para ver el resto. Esto es lo que significa cada uno:

CAMPO	QUÉ SIGNIFICA	ERROR TÍPICO AL LEERLO
Transaction Hash	El identificador: el hash de la transacción firmada. Es lo que se pega en el informe.	Copiar el hash del bloque en vez del de la transacción.
Status	Success o Fail .	Creer que un Fail no costó nada.
Block	Número del bloque que la incluyó y cuántas confirmaciones lleva encima.	Confundir confirmada con finalizada (Sesión 4).
From	Quien firmó. Siempre una cuenta externa (EOA).	—
To	La cuenta o el contrato al que se dirigió. Un ícono de documento indica contrato.	En un token, To es el CONTRATO del token, no quien recibe.
Value	ETH enviado junto con la llamada.	Pensar que 0 ETH significa que no se movió nada.
Transaction Fee	Lo que pagó el remitente en total por el gas.	Confundirlo con el valor transferido.
Gas Price	El precio efectivo por unidad de gas: tarifa base + propina, sin pasar del máximo.	Usar el máximo firmado para calcular el costo.
Gas Limit & Usage by Txn	El tope firmado y lo que se consumió, con el porcentaje.	Creer que se cobra el límite completo cuando terminó bien.
Gas Fees	Base (tarifa base del bloque), Max (maxFeePerGas firmado) y Max Priority (propina máxima firmada).	Creer que la base la eligió el usuario.
Burnt & Txn Savings Fees	Burnt: lo quemado (gas usado × base). Savings: lo que se firmó como máximo y no se cobró.	Creer que lo quemado lo recibió el validador.
Nonce	El contador de la cuenta remitente para esta transacción.	—
Input Data	El campo <code>data</code> : selector de 4 bytes y argumentos. Con <i>Decode Input Data</i> se ve la función y sus valores.	No ver nada útil porque el contrato no está verificado.
Pestaña Logs	Los eventos que emitió la ejecución.	Ignorarlos: son la forma más confiable de saber qué pasó.

05.3 · El Input Data, decodificado a mano

El explorador decodifica solo si el contrato está verificado. Pero ustedes pueden hacerlo a mano, y el informe lo pide al menos una vez. Tomen el *Input Data* en vista *Original* (hexadecimal) y pártanlo así:

[illegible]

TROZO	CÓMO SE LEE
Selector	<code>a9059cbb</code> son los primeros 4 bytes de <code>keccak256("transfer(address,uint256)")</code> . El mismo selector en cualquier token ERC-20 del mundo.
Argumento 1	La dirección de destino: los últimos 20 bytes (40 caracteres). Los 24 ceros de la izquierda son relleno para completar 32 bytes.
Argumento 2	El monto, en hexadecimal y en la unidad mínima del token. <code>0x1142f20</code> = 18 100 000. USDC tiene 6 decimales: $18\ 100\ 000 \div 10^6 =$ 18,1 USDC . LINK tiene 18.

Para convertir hexadecimal a decimal: en la consola del navegador (F12 → *Console*) escriban `BigInt("0x1142f20")` y Enter. Para saber los decimales de un token: en la página del contrato, pestaña *Contract* → *Read Contract* (o *Read as Proxy*) → `decimals`.

05.4 · Los logs, decodificados

Un evento `Transfer` de un token aparece en la pestaña *Logs* con esta forma:

PARTE DEL LOG	QUÉ CONTIENE
Address	El contrato que emitió el evento. Tiene que ser el token esperado: un evento de otro contrato no prueba nada.
Topics[0]	0xddf252ad...b3ef = keccak256("Transfer(address,address,uint256)"): la firma del evento.
Topics[1] · [2]	Los parámetros indexed : origen y destino, rellenos a 32 bytes.
Data	Los parámetros no indexados: el monto. Debe coincidir con el argumento 2 del Input Data.

Comprobar que el monto del log coincide con el del Input Data es exactamente el tipo de verificación cruzada que hace un auditor.

05.5 · Las cuentas: descomponer el costo

Con EIP-1559 (láminas A.13 a A.15), para cada transacción:

```

precio efectivo = min( Max , Base + Max Priority )
Transaction Fee = Gas Used × precio efectivo
Burnt           = Gas Used × Base
propina         = Gas Used × (precio efectivo - Base) = Fee - Burnt
no cobrado     = Gas Used × (Max - precio efectivo)

```

Trabajen en **gwei** y conviertan al final: 1 gwei = 0,000000001 ETH. Escriban la multiplicación completa en el informe; la calculadora del computador sirve. **La suma Burnt + propina tiene que dar exactamente el Fee.** Si no da, algo se copió mal.

EJEMPLO RESUELTO · TRANSFERENCIA DE USDC · SEPOLIA · BLOQUE 11 727 240

Hash `0xe51f75bad82aceb9e03cbf9af64d080df91203c43f801b99f7d71556654fbf0a` , consultada el 17 de septiembre de 2026. Es el modelo de precisión que se espera; no la usen en su informe.

CAMPO	VALOR
Tipo · Status	Transferencia de token (llamada a contrato) · Success
To · ¿persona o contrato?	<code>0x1c7D...7238</code> · contrato (USDC, detrás de un proxy)
Value	0 ETH
Gas Limit · Gas Used	69 301 · 62 171 (89,7 % del límite)
Base · Max · Max Priority	0,961679578 · 1,158789515 · 0,0011 gwei
Precio efectivo	$\min(1,158789515 ; 0,961679578 + 0,0011) = 0,962779578$ gwei
Transaction Fee	$62\ 171 \times 0,962779578$ gwei = 0,000059856969143838 ETH
Burnt	$62\ 171 \times 0,961679578$ gwei = 0,000059788581043838 ETH
Propina	$62\ 171 \times 0,0011$ gwei = 0,0000000683881 ETH · comprobación: Burnt + propina = Fee ✓
Selector	<code>0xa9059cbb</code> · <code>transfer(address,uint256)</code>
Argumentos	destino <code>0x9d7f...25db</code> · monto 18 100 000 = 18,1 USDC
Logs	1 evento <code>Transfer</code> emitido por el contrato de USDC, con el mismo destino y el mismo monto ✓

Lo que un buen informe añade: el bloque 11 727 240 usó el 99,98 % de su límite; por eso, según la fórmula de EIP-1559, la tarifa base del bloque siguiente subió de 0,961679578 a 1,081844398 gwei (+12,5 %). La propina fue casi nula porque Sepolia tiene poca competencia por espacio.

EJEMPLO DE UNA TRANSACCIÓN FALLIDA · BLOQUE 11 727 244

Hash `0xcb485277658bbe9d1d9133078f1f9ea543a1b9091fa03a3fdbf68d055cf278da` . El remitente llamó a `transfer` en un token «confidencial» que no admite esa función. El explorador muestra `Fail With Custom Error 'FHERC20IncompatibleFunction()'` . Firmó un límite de 500 000; la ejecución revirtió tras consumir **22 550** de gas y pagó $22\ 550 \times 1,947200913$ gwei = **0,0000439 ETH** por un resultado nulo. No hay logs: una ejecución revertida no deja eventos.

05.6 • Las cuatro preguntas que el informe responde

1. **Transacción 1:** ¿el gas usado es exactamente 21 000? Si no lo es, ¿qué indica sobre el destinatario o sobre la red? (Vean la caja de la sección 07.)
2. **Transacción 2:** el campo *Value* dice 0 ETH. ¿Dónde está entonces el monto transferido? Muéstrenlo en el *Input Data* decodificado a mano y en el evento `Transfer`, y comprueben que coinciden. ¿El selector es `0xa9059cbb`? Si no, ¿qué función se llamó?
3. **Transacción 3:** ¿por qué falló? ¿Cuánto gas consumió frente al límite firmado? ¿Cuánto pagó el remitente a pesar del fallo? ¿Qué quedó en los logs?
4. **Las tres:** comprueben a mano que `Fee = Gas Used × precio efectivo` y que `Burnt + propina = Fee`. Escriban las multiplicaciones.

06 Parte B • Predecir y medir el gas

25 MINUTOS

El instrumento es el contrato `contracts/s05/Operaciones.sol`: cada función hace **una sola cosa costosa**, para poder aislar su gas. No es un contrato útil: es un aparato de medición.

1

10 MINUTOS • SIN CORRER NADA

Predecir

Abran `contracts/s05/Operaciones.sol` en su editor y lean cada función. Con la tabla de la lámina A.17 escriban su predicción en la columna correspondiente. **Se evalúa el razonamiento, no el acierto.** Anoten al lado de cada número de dónde salió («21 000 de base + 22 100 de SSTORE nuevo»).

Recuerden sumar siempre: 21 000 de base + el costo de los datos de la llamada (4 por byte en cero, 16 por byte distinto de cero; una llamada con un `uint256` pequeño lleva 36 bytes: 4 del selector y 32 del número) + las instrucciones caras + un pequeño resto que pone Solidity.

OPERACIÓN	PREDICCIÓN	MEDICIÓN	DIFERENCIA	¿POR QUÉ?
Transferir ETH sin contrato				
Desplegar el contrato				
Escribir en una ranura vacía (0 → 42)				
Sobrescribir una ranura ocupada (42 → 43)				
Leer la ranura, dentro de una transacción				
Borrar la ranura (43 → 0)				
Emitir un evento con dos datos				
Sumar 50 números · memory				
Sumar 50 números · calldata				
Primer elemento de un arreglo				
Agregar 1 elemento más				
Agregar 10 elementos más				

2

2 MINUTOS · COMPROBAR EL ENTORNO

Correr las pruebas de la sesión

En la carpeta `laboratorios-evm`:

```
npx hardhat test test/s05/Operaciones.test.js
```

Salida esperada: compila los contratos (la primera vez tarda) y muestra siete líneas con ✓, terminando en 7 `passing`. Las pruebas no verifican cifras exactas sino las **relaciones** que la clase afirma: que una transferencia simple cuesta 21 000, que estrenar una ranura es mucho más caro que sobrescribirla, que un evento es más barato que escribir, que calldata es más barato que memory.

Cómo sé que funcionó: 7 `passing` y ningún `failing`.

Correr el script de medición

```
npx hardhat run scripts/s05/medir-gas.js
```

El script despliega `Operaciones` en la **red local de Hardhat** (dentro de su computador: no gasta ETH de prueba ni necesita internet), ejecuta cada operación y lee el `gasUsed` de cada recibo. **Salida esperada** (Hardhat 3.16, Solidity 0.8.28, optimizador con 200 ejecuciones):

OPERACIÓN	GAS USADO
Transferir ETH entre cuentas (sin contrato)	21.000
Desplegar el contrato	271.161
Escribir en una ranura vacía (0 → 42)	43.602
Sobrescribir una ranura ocupada (42 → 43)	26.479
Leer la ranura, dentro de una transacción	23.364
Borrar la ranura (43 → 0)	21.445
Emitir un evento con dos datos	22.862
Sumar 50 números · parámetro en memory	42.058
Sumar 50 números · parámetro en calldata	39.360
Primer elemento de un arreglo (estrena el largo)	65.895
Agregar 1 elemento más	48.795
Agregar 10 elementos más	251.250

Con 2 gwei por unidad de gas, escribir una ranura nueva cuesta 0.000087204 ETH.

Cómo sé que funcionó: la tabla sale completa, con 12 filas. Si sus cifras difieren en unas pocas unidades, revisen la versión del compilador; las proporciones deben mantenerse.

Explicar las dos diferencias más grandes

Copien las mediciones a su tabla, calculen la diferencia y elijan las **dos mayores** (en valor absoluto). Para cada una, escriban qué no tuvieron en cuenta. Pistas, sin dar la respuesta:

- La **base de 21 000** se paga en toda transacción: réstenla antes de comparar con la tabla de instrucciones.
- El **primer acceso** a una ranura dentro de una transacción cuesta 2 100 adicionales (acceso «frío», EIP-2929).
- **Borrar** una ranura (volverla a cero) genera un reembolso de 4 800, con tope de un quinto del gas usado (EIP-3529).
- El **primer** elemento de un arreglo estrena además la ranura donde se guarda su largo.
- Los **argumentos** también cuestan: 4 por byte en cero y 16 por byte distinto de cero.
- Las dos filas de «sumar 50 números» se miden con `estimateGas`: el arreglo viaja en la llamada, y copiarlo a memoria tiene precio.

Escribir en una ranura vacía = 43 602. Base 21 000 + datos 204 (el selector y el número 42 ocupan 36 bytes: 31 en cero $\times 4 = 124$ y 5 distintos de cero $\times 16 = 80$) + `SSTORE` de cero a un valor, con primer acceso: 20 000 + 2 100 = 22 100 + resto de Solidity 298 (comparar el selector, decodificar el argumento, saltar a la función). Total: 21 000 + 204 + 22 100 + 298 = 43 602 ✓.

07 Puntos de control

¿VOY BIEN?

MINUTO	DEBERÍAN TENER
5	Billetera en Sepolia, con saldo, y su dirección abierta en el explorador.
15	Transacción 1 encontrada y su hoja de trabajo llena, con la suma Burnt + propina = Fee comprobada.
25	Transacción 2 con el Input Data partido en selector y argumentos, y el monto del log igual al del argumento.
35	Transacción 3 con el motivo del fallo anotado. Empiezan la parte B.
45	Tabla de predicciones completa, sin haber corrido el script.
55	<code>7 passing</code> y la tabla de mediciones copiada. Diferencias calculadas.

SI SEPOLIA YA ACTIVÓ LA ACTUALIZACIÓN GLAMSTERDAM

A la fecha de esta guía estaba prevista para Sepolia el 6 de octubre de 2026, y cambia varios precios: entre otros, baja el costo base de una transacción simple y sube el de crear estado. Si su transacción 1 no gastó 21 000, **no es un error de ustedes**: anoten el valor real, averigüen en qué fecha se activó la actualización en Sepolia y úsenlo para responder la pregunta 1. La red local de Hardhat de la parte B conserva los precios de la tabla de la clase, y comparar las dos redes es un hallazgo que suma.

SÍNTOMA	CAUSA Y SOLUCIÓN
El explorador no encuentra mi dirección o mi hash	Están en <code>etherscan.io</code> (red principal). Usen <code>sepolia.etherscan.io</code> .
«Value: 0» en una transferencia de token	Correcto: el token no es ETH. El monto está en el Input Data y en los logs.
El Input Data es ilegible y no hay botón Decode	El contrato no está verificado. Decodifiquen a mano (05.3) o elijan uno de los dos tokens sugeridos.
El monto del token parece absurdo (millones)	No dividieron por los decimales. USDC usa 6, LINK usa 18. Consulten <code>decimals</code> en Read Contract.
Burnt + propina no da el Fee	Usaron <i>Max</i> en vez del precio efectivo, o copiaron la base de otro bloque. Rehagan con 05.5.
No encuentro ninguna transacción fallida	Cambien de página en la lista del contrato o prueben con otro contrato activo de un bloque reciente.
La transacción 1 no gastó 21 000	El destinatario es un contrato (tiene código y ejecutó algo), o Sepolia ya cambió sus precios (caja de la sección 07).
<code>node</code> o <code>npx</code> no se reconoce	Node.js no está instalado o la terminal se abrió antes de instalarlo. Instalen Node 22 LTS y abran otra terminal.
<code>npm install</code> falla en la sala	Restricción de red. Plan B: el docente comparte la carpeta <code>node_modules</code> por red local.
Error que menciona «No Hardhat config file found»	La terminal no está en la carpeta <code>laboratorios-evm</code> . Usen <code>cd</code> para entrar.
Las cifras del script difieren un poco de la guía	Otra versión del compilador u optimizador. Revisen <code>versiones.md</code> ; las proporciones deben mantenerse.
La billetera pide firmar algo	Hoy no se firma nada. Rechacen y avisen al docente.

Estas preguntas se responden con **sus** transacciones y **sus** mediciones. Una respuesta genérica, sin cifras propias, no puntúa. Elijan al menos dos para el informe, además de la reflexión obligatoria.

1. En su transacción 2, ¿qué porcentaje del límite firmado se usó? Si el límite lo propuso una billetera, ¿qué margen dejó sobre la estimación? ¿Qué habría pasado con un límite 10 % menor que el gas usado?
2. Busquen la tarifa base del bloque de su transacción 1 y la del bloque siguiente. Con la fórmula de la lámina A.14, ¿el cambio corresponde a un bloque más lleno o más vacío que la mitad? Compruébenlo mirando el *Gas Used* del bloque.
3. En su transacción fallida, ¿cuánto del límite firmado quedó sin cobrar? ¿Por qué el protocolo cobra el gas consumido aunque el estado no cambie?
4. En su medición, ¿cuántas veces más caro es «agregar 10 elementos» que «agregar 1»? ¿Por qué no es exactamente 10?
5. Con su medición de «escribir en una ranura vacía» y la tarifa base de su transacción 1, ¿cuánto costaría en Sepolia guardar 1 000 números nuevos, uno por transacción? ¿Qué cambiarían en el diseño?
6. **Reflexión obligatoria:** después de hoy, ¿qué operación evitarían en un contrato real y qué harían en su lugar?

10 Plantilla del informe corto de evidencia

Copien esta estructura en su documento. Las tablas pueden ir en letra pequeña; lo que no puede faltar son los hashes y las cuentas.

```

INFORME · LABORATORIO 05 · FORENSE Y GAS
Pareja: Nombre 1 · Nombre 2                      Fecha: DD/MM/AAAA
Explorador: sepolia.etherscan.io                  Consultado: DD/MM/AAAA

1. TRANSACCIONES ANALIZADAS
T1 · transferencia de ETH · https://sepolia.etherscan.io/tx/0x...
T2 · transferencia de token · https://sepolia.etherscan.io/tx/0x...
T3 · transacción fallida · https://sepolia.etherscan.io/tx/0x...

2. HOJA DE TRABAJO (una columna por transacción)
Tipo · Status · Bloque · From · To (¿persona o contrato?) · Value
Gas Limit · Gas Used (% del límite)
Base · Max · Max Priority · precio efectivo (gwei)
Transaction Fee · Burnt · propina · no cobrado (ETH)
Comprobación: Burnt + propina = Fee [escribir la suma]
Selector · función · argumentos decodificados
Logs: evento, emisor, monto

3. LAS CUATRO PREGUNTAS (sección 05.6), con cifras

4. GAS: tabla de 12 filas (predicción · medición · diferencia · por qué)
Las dos diferencias mayores, explicadas en 3-5 líneas cada una

5. REFLEXIÓN (un párrafo) + al menos dos preguntas de la sección 09

6. DECLARACIÓN DE USO DE IA: para qué se usó, o «no se usó»

```

11 Lista de chequeo antes de entregar

✓	COMPROBACIÓN
☐	Los tres hashes abren en <code>sepolia.etherscan.io</code> (probado el día de la entrega, desde otro navegador o en modo incógnito).
☐	Las tres transacciones son de tipos distintos y ninguna es de los ejemplos de la guía o de las láminas.
☐	En las tres, Burnt + propina = Fee, con la suma escrita.
☐	La transacción 2 tiene el Input Data partido a mano y el monto convertido con los decimales correctos.
☐	La transacción 3 tiene el motivo del fallo y cuánto pagó.
☐	La tabla de gas tiene las 12 predicciones escritas antes de medir, y las 12 mediciones.
☐	Las dos mayores diferencias están explicadas con la descomposición (base + datos + instrucciones + resto).
☐	Hay reflexión, al menos dos preguntas de la sección 09 y la declaración de uso de IA.
☐	Dos páginas como máximo. Ningún dato de la billetera más allá de la dirección pública. Ninguna frase ni clave.

12 Rúbrica

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (<3,0)
A · lectura y costos	30 %	Tres transacciones verificables de tipos distintos, todos los campos, sumas exactas.	Algún campo o suma con errores menores.	Hashes que no abren, tipos repetidos o cuentas sin hacer.
A · data y logs	20 %	Input Data partido a mano, decimales correctos, monto cruzado con el log, motivo del fallo explicado.	Solo la versión decodificada por el explorador.	No identifica selector ni monto.
B · predicción y medición	35 %	Predicciones razonadas antes de medir; las dos mayores diferencias explicadas con la descomposición.	Predicciones sin razonamiento o explicación vaga.	Predicciones copiadas de la medición o faltantes.
Reflexión	15 %	Responde con cifras propias y propone una alternativa de diseño concreta.	Correcta pero genérica.	Ausente o sin relación con los datos.

13 Referencias

- Buterin, V. et al. (2019). *EIP-1559: Fee market change for ETH 1.0 chain*.
<https://eips.ethereum.org/EIPS/eip-1559>
- Buterin, V. y Swende, M. (2020). *EIP-2929: Gas cost increases for state access opcodes*.
<https://eips.ethereum.org/EIPS/eip-2929>
- Buterin, V. y Swende, M. (2021). *EIP-3529: Reduction in refunds*. <https://eips.ethereum.org/EIPS/eip-3529>
- Referencia interactiva de instrucciones y costos: <https://www.evm.codes/>
- ethereum.org. *Gas and fees*. <https://ethereum.org/developers/docs/gas/>
- ethereum.org. *Glamsterdam* (estado de la actualización). <https://ethereum.org/roadmap/glamsterdam/>
- Explorador de Sepolia: <https://sepolia.etherscan.io>